

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

“Д34: Технический дизайн микросервисной архитектуры”

Выполнил:

Данилова Анастасия

Группа

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы их взаимодействия друг с другом
3. Спроектировать разделение БД на обособленные БД
4. Спроектировать и описать эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Описать варианты запросов и ответов, задокументировать возможные ошибки
6. В результате сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения монолитного бэкенд-приложения на отдельные микросервисы

Цель работы

Целью работы является проектирование микросервисной архитектуры для ранее разработанного монолитного backend-приложения сервиса аренды недвижимости.

В рамках работы необходимо:

- разделить монолитное приложение на независимые микросервисы;
- реализовать концепцию database-per-service;
- определить зоны ответственности каждого сервиса;
- спроектировать взаимодействие между микросервисами;
- описать REST API для межсервисного взаимодействия;
- продумать форматы запросов, ответов и возможные ошибки API.

Постановка задачи

В рамках домашнего задания необходимо выполнить техническое проектирование микросервисной архитектуры для backend-приложения сервиса аренды недвижимости.

Требуется:

- выполнить декомпозицию монолитного приложения на отдельные микросервисы;

- определить состав и назначение каждого микросервиса;
- спроектировать отдельные базы данных для каждого сервиса согласно принципу database-per-service;
- определить способы взаимодействия между сервисами;
- описать REST API межсервисного взаимодействия;
- задокументировать структуры запросов и ответов;
- определить возможные ошибки и коды ответов API.

Результатом работы должен стать документ, содержащий описание архитектуры системы, взаимодействия сервисов и спецификацию API.

Анализ исходного монолитного приложения

Изначально backend-приложение сервиса аренды недвижимости было реализовано в виде монолитной архитектуры.

Приложение представляло собой единый backend-сервис, содержащий:

- общую бизнес-логику;
- единое REST API;
- одну общую базу данных;
- централизованную обработку запросов.

В монолитном приложении были реализованы следующие сущности:

- пользователи;
- недвижимость;
- аренда;
- чаты;
- сообщения;
- изображения недвижимости;
- удобства объектов недвижимости.

Основные функции системы:

- регистрация и авторизация пользователей;
- управление объектами недвижимости;
- создание и просмотр аренд;
- обмен сообщениями между пользователями;
- хранение изображений объектов;
- фильтрация недвижимости.

Несмотря на простоту разработки и развертывания монолитной архитектуры, подобный подход обладает рядом недостатков:

- сложность масштабирования отдельных компонентов системы;
- высокая связность бизнес-логики;
- усложнение сопровождения проекта при увеличении количества функциональности;
- невозможность независимого развертывания отдельных частей системы;
- увеличение риска отказа всего приложения при ошибке в одном из компонентов.

Для решения данных проблем было принято решение о переходе к микросервисной архитектуре с разделением системы на независимые сервисы.

Выделение микросервисов

В результате анализа предметной области и существующего монолитного приложения система была разделена на несколько независимых микросервисов.

Разделение выполнялось по принципу business domain decomposition — каждый сервис отвечает за отдельную бизнес-область и обладает собственной базой данных.

В результате были выделены следующие микросервисы:

- Auth Service;
- Property Service;
- Rental Service;
- Chat Service.

Auth Service

Назначение

Auth Service отвечает за управление пользователями и аутентификацию в системе.

Сервис обеспечивает:

- регистрацию пользователей;
- авторизацию;
- генерацию JWT-токенов;

- проверку токенов;
- получение информации о пользователях.

Основные сущности

- User

Зона ответственности

Auth Service отвечает исключительно за:

- учетные записи пользователей;
- безопасность;
- аутентификацию;
- авторизацию.

Другие сервисы не хранят пользовательские данные полностью, а обращаются к Auth Service через HTTP API.

Основные функции

- регистрация пользователя;
- вход в систему;
- выдача JWT-токена;
- получение профиля пользователя;
- проверка существования пользователя.

Property Service

Назначение

Property Service отвечает за работу с объектами недвижимости.

Сервис управляет:

- созданием объектов недвижимости;
- хранением описаний объектов;
- изображениями объектов;
- удобствами недвижимости;
- поиском и фильтрацией.

Основные сущности

- Property
- PropertyImage
- Amenity

Зона ответственности

Property Service отвечает исключительно за данные, связанные с недвижимостью:

- описание объектов;
- стоимость аренды;
- местоположение;
- фотографии;
- удобства;
- статус доступности объекта.

Основные функции

- создание объекта недвижимости;
- получение списка объектов;
- фильтрация недвижимости;
- обновление информации об объекте;
- загрузка изображений;
- управление удобствами.

Rental Service

Назначение

Rental Service отвечает за процессы аренды недвижимости и управление сделками.

Сервис обеспечивает:

- создание аренд;
- хранение информации о бронированиях;
- изменение статусов аренды;
- получение истории аренд.

Основные сущности

- Rental

Зона ответственности

Rental Service отвечает исключительно за:

- аренду недвижимости;
- бронирования;
- сроки аренды;
- статусы сделок;
- связь арендаторов с объектами недвижимости.

Основные функции

- создание аренды;
- просмотр списка аренд;
- изменение статуса аренды;
- отмена аренды;
- получение истории сделок пользователя.

Chat Service

Назначение

Chat Service обеспечивает обмен сообщениями между пользователями системы.

Сервис используется для общения:

- арендаторов;
- владельцев недвижимости.

Основные сущности

- Chat
- Message

Зона ответственности

Chat Service отвечает исключительно за:

- создание чатов;
- хранение сообщений;
- получение истории переписки;
- передачу сообщений между пользователями.

Основные функции

- создание чатов;
- отправка сообщений;
- получение списка чатов пользователя;
- получение истории сообщений;
- хранение переписки между пользователями.

Проектирование database-per-service

В микросервисной архитектуре был реализован подход database-per-service, согласно которому каждый сервис обладает собственной независимой базой данных.

Подобный подход позволяет:

- уменьшить связанность между сервисами;
- обеспечить независимое масштабирование;
- повысить отказоустойчивость;
- изолировать бизнес-логику отдельных компонентов системы.

Каждый микросервис взаимодействует только со своей базой данных и не имеет прямого доступа к БД других сервисов.

Распределение баз данных между сервисами

Микросервис	База данных	Таблицы
Auth Service	auth_db	users
Property Service	property_db	properties, property_images, amenities
Rental Service	rental_db	rentals
Chat Service	chat_db	chats, messages

Отсутствие внешних ключей между сервисами

В рамках микросервисной архитектуры не используются прямые foreign key между таблицами различных сервисов.

Например:

- Rental Service хранит только user_id и property_id;
- Chat Service хранит только идентификаторы пользователей и объектов недвижимости.

Проверка существования связанных сущностей выполняется через HTTP REST API межсервисного взаимодействия.

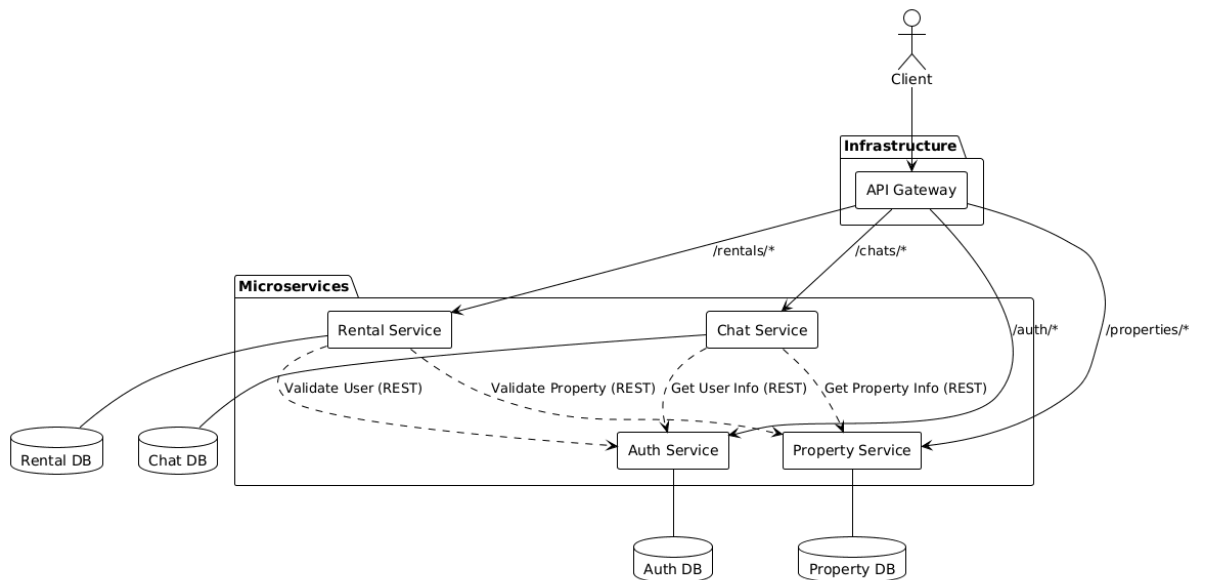
Данный подход позволяет:

- избежать сильной связанности между сервисами;
- обеспечить независимость баз данных;
- упростить масштабирование и развертывание отдельных компонентов системы.\

Диаграмма database-per-service

Архитектура взаимодействия микросервисов

Для обеспечения единой точки входа и безопасности используется API Gateway. Взаимодействие между сервисами осуществляется по протоколу HTTP/REST.



Проектирование межсервисного взаимодействия

Для обеспечения взаимодействия между микросервисами была спроектирована система внутренних REST API согласно принципам микросервисной архитектуры и подходу database-per-service.

Каждый сервис обладает собственной изолированной базой данных и не обращается напрямую к таблицам других сервисов. Получение связанных данных осуществляется исключительно через внутренние HTTP API (`/internal/* endpoints`).

Взаимодействие между сервисами реализовано следующим образом:

- **Rental Service** → **Property Service** - получение информации об объекте недвижимости, проверка существования объекта, статуса доступности и владельца;
- **Rental Service** → **Auth Service** - получение информации о пользователе и валидация tenant;

- Chat Service → Property Service - получение информации об объекте недвижимости для создания чата;
- Chat Service → Auth Service - получение информации об участниках чата;
- Property Service → Auth Service - валидация владельца недвижимости.

Для каждого микросервиса была разработана спецификация OpenAPI 3.0, включающая:

- описание публичных REST-endpoints;
- описание внутренних endpoints для межсервисного взаимодействия;
- схемы входящих запросов (request body);
- схемы ответов (response body);
- описание HTTP-статусов;
- документирование ошибок (400, 401, 403, 404, 409, 500);
- JWT-аутентификацию и security schemes.

Таким образом, требования задания по проектированию API, документированию запросов, ответов и ошибок были реализованы посредством единого формального описания в формате OpenAPI Specification.

Вывод

В ходе работы была успешно спроектирована микросервисная архитектура для системы аренды недвижимости. Монолитное приложение было разделено на 4 функциональных сервиса. Реализован принцип database-per-service, что позволило устранить жесткие связи на уровне базы данных. Описанные спецификации API и схемы взаимодействия обеспечивают консистентность данных и готовность системы к горизонтальному масштабированию.